Ezma Babadilla-Cruz Homework # 8

Q1)

Q1) Goal: maximize the Number of balls we are able to store in boxes.
We can assume we are provided 2 arrays such that D[1...n] where D[i] = diameter of ith ball
and L[1...m] where L[j] = length of jth box

a) develop a Greedy Alg. Pseudo code

<u>Store balls</u>

1) Sort out arrays D and L in descending order (descending order of ball diameters and box lengths) using mergeSort such that TC is O(nlogn) + O(mlogm)

```
//Note that n = length of D[] and m = length of L[]
    i, j = 0   // we will use indicies to iterate
    NumballsStored = 0 // keep track
while(D[i] > L[j] && i ≤ n)   // want to make sure we are within bounds but only want to iterate at MOST n times
        // diameter is larger than box length and this is the largest Possible box
        since we sorted our arrays in descending order, so skip this ball
        i++
//outside while, meaning D[i] ≤ L[j]
    if( i ≤ n && j ≤ m) //check bounds, we have to make sure we have a valid box
        Position
    //store in this box
        L[j] = D[i]
    //increment position
        j++
        i++
        NumballsStored++

return NumballsStored
```

Part One

b) Time Complexity = O(nlogn) + O(mlogm). The first part of our alg sorts arrays D[1...n] and L[1...m] using mergeSort so TC = O(nlogn) + O(mlogm)
Part Two: we want to iterate at most n times thus the TC is $\Theta(n)$

Total TC = O(nlogn) + O(mlogm) + $\Theta(n)$

Ethan Cruz

Q2

Rewrite BFS using adjMatrix, //assume adjacency Matrix is called adjM
BFS (G): for vertices 1 to n in V[G] if (S.l == undiscovered) → BFS (G,S)

```
BFS (G,S)
    queue Q
    Q.push(S) //push source vertex
    S.label = seen
    While Q is not empty
        u = Q.Front()
        Print u
        Q.Pop() //remove u

//visit every vertex   for vertex in adjM (v=1 to n]   //We can have up to n vertices
            if (adjM [u, v] == 1 && v.label==undiscovered)
//push vertex to queue   Q.push(V)
                        adjM.label=seen
//adjM[i] = vertexNum    adjM.parent = u
//
```

Checking time and space Complexity:

- TC: $O(|V|^2) \simeq \Theta(n^2)$ //size of 2D array
- Space Comp $= \Theta(|V|) \simeq \Theta(n)$ (for queue)

Ezra B Cruz

Q3) Q3

Q3) given undirected graph $G=(V,E)$, we want to check if its connected within $O(V+E)$ time

Def of connected graph: there is a walk b/w any 2 vertices in graph

def of undirected graph: Simplest connected graph

Thus, this means all vertices within our graph MUST have a walk/we have "discovered" every vertex if we run BFS once.

Pseudo Code // Assume we implement BFS using adj Lists (Linked Lists)

```
isConnected (G)

    BFS(G)

    for vertex = 1 to n (in V(G))
        if vertex.label == Undiscovered
            return False // Not connected

    return True // we checked all vertices are connected
```

Time Complexity:

Part 1: BFS running time = $O(|V|+|E|)$

Part 2: for loop up to the number of vertices = $O(|V|)$

Thus total TC = $O(|V|+|E|) + O(|V|)$ which is asymptotically equal to $O(|V|+|E|)$

Ezra BCruz

Q4

4) G is a directed graph on n vertices, we are given adj Matrix V=[1..n][1..n]

Develop O(n) time alg to check if there is a vertex that has edges coming to it from all other vertices but no edges going out

Note: If there are no edges going out at vertix n then v[n,] has to equal zero so that row n is all zeros since all other vertices No edges are coming out

Pseudo Code

```
// we will traverse through the adj Matrix starting from right

i,j = 1   // index starts at 1

while(i ≤ n && j ≤ n)   // within bands, haven't reached the end of adj Matrix
    if V[i][j] == 0
        j++   // move to the right of this array because we want v[i][i] == 1
    else // we found an edge going out so we want to see if this vertex is our answer
        i++   // move down since v[i][j] == 1 to check if any edges coming in
                                                                  TC = O(n)

//check if we traversed through the whole adj Matrix
if j ≤ n (we didn't find answer)
    return false   // We did not find such vertex in adj M

// want to make sure that we have all zeros in the ith row
for i = 1 to n
    if v[i][i] == 1   //
        return false
                                                                  TC = O(n)
return true // we have all zeroes in this row, this is our vertex (V[i])
```

Time Complexity:

TC = 2O(n) ≃ O(n). As highlighted/Noted Noted above, At most our TC=2O(n) which is asymptotically equal to O(n).